

Model Card — AOT-GOAT (Recipient-Aware Message Adaptation)

Build–Audit–Pitch mini NLP project · Research prototype · Values verified against github.com/painsparc/AOT-GOAT

1. Model details

Model name	AOT-GOAT (AOT = decision gate, GOAT = minimal-intervention selection)
Task	Detect recipient-conditioned ambiguity in a draft message (given sender–recipient history) and, only if needed, make the smallest evidence-grounded edit. Five risk types: coreference, temporal, scope, terminology, implicit context.
Architecture	Deterministic rule / lexicon / heuristic pipeline in Python (not a trained neural model; no training, no learned weights): history → dyadic profile → rule-based risky spans → risk score → AOT pass/intervene → GOAT greedy edit set → template replacement → hard-constraint check. Optional LLM adapter (Anthropic / OpenAI-compatible) can propose extra spans and rephrase; scoring, decision and constraints stay rule-based. Offline mode used for all reported results.
Risk score	risk = base × (1 – grounding) × (1 + 0.5 × clarification_rate). grounding = how much the dyad's history already covers the span; clarification_rate = recipient's past clarification questions of that type (capped at 1).
Thresholds (CFG)	tau_span 0.40 · tau_high 0.55 (noisy-OR message risk) · tau_low 0.30 · noise floor 0.25 · residual factor 0.15 · max 14 added tokens per edit. Intervene if any span ≥ 0.40 or message risk ≥ 0.55.
Data & preprocessing	No training data. Evaluation set: data/eval_set.json, 59 synthetic, author-written cases (41 dev, 18 holdout); each has a message, S↔R history, optional sender context / time zones and gold risk types. Preprocessing: history strings "S: ... / R: ..." normalised; lowercase regex tokenisation [a-z0-9]+; stop-word and common-acronym lists; regex lexicons for dates, jargon, scope and anchor phrases; time-zone parsing (UTC±h, IST/EST/PST...).
Software	Python 3.9+ (developed on 3.12); flask ≥3.0,<4 · requests ≥2.31 · python-dotenv ≥1.0 · gunicorn (non-Windows). No ML libraries. Deterministic offline output. Developed by: painsparc (GitHub) · NLP course project, October 2026.

2. Intended use

- Intended:** research / educational prototype that helps a sender writing to a recipient who lacks shared context (e.g., a new contact in another time zone) by flagging ambiguous spans and applying minimal edits.
- Out of scope:** production communication-safety use; non-English text; paraphrase-level or unmarked implicit context; any claim of validated real-world accuracy or fairness; unattended public deployment with an LLM key (no authentication or rate limiting).

3. Overall performance (offline analyzer, python evaluate.py)

Level / split	Precision	Recall	F1	Notes
Risk-type, dev (n=41)	0.929	1.000	0.963	Micro-averaged over (case, risk type) labels; predicted positive = span of that type with risk ≥ 0.40
Risk-type, holdout (n=18)	1.000	0.929	0.963	Holdout written after tuning, never used for tuning (same author and style)
Risk-type, overall (n=59)	0.951	0.975	0.963	
Decision (PASS vs INTERVENE), n=59	0.969	0.969	0.969	TP 31 · TN 26 · FP 1 · FN 1 · accuracy 57/59 = 96.6% · FPR 3.7% · FNR 3.1%
Baseline: always INTERVENE (derived from the counts above)	0.542	1.000	0.703	Accuracy 32/59 = 54.2% (32 of 59 cases gold-intervene); system gains +42.4 points accuracy

Constraint checks on all 13 adapted outputs: intent preserved 13/13 · no invented facts 13/13 · minimal edit 13/13 (partly by construction: the checker gates the output). Checker self-test with injected violations (invented name, dropped negation, over-long edit, lost meaning): 13/13 detected for each. Predicted intervention rate 32/59 = 54.2%. Test suite: 28 end-to-end tests (LLM mocked).

4. Sample predictions and behaviour across recipients / error analysis

Message: *“Please send it to everyone by EOD tomorrow, as discussed. The SLA needs sign-off before the meeting.”* — the same text, three different recipient histories (README demo):

Recipient (history)	Decision	Output
Asha — same project and time zone, has seen all of it	PASS	Message returned unchanged.
Vikram — new contact, 10.5 h time-zone gap, nothing shared	INTERVENE	All six detected spans edited (grounded in sender-supplied context, history, computed dates).
Meera — knows the report and the EOD convention, not “everyone” or “SLA”	INTERVENE	Only <i>it, everyone, SLA</i> edited; remaining text kept verbatim.
API example: “Please share your feedback soon.” (neutral history)	Needs sender input	“soon” is vague with no supporting evidence: left as written and the sender is asked for an exact date/time (no guessing).

Known errors: dev (2) — idiom “Everything is all set” flagged; Meera’s *it* over-flagged. Holdout (1) — miss on “Let’s go with what we did for Mumbai” (implicit context with no marker phrase). Per-risk-type and per-context subgroup tables are printed by evaluate.py, but row-level outputs were not retained in this card, so they are not reported here.

5. Explainability

Intrinsic, pipeline-level: every output lists each span with its **type, reason, base severity, grounding score and grounding note (history evidence), recipient clarification rate, risk, AOT decision, GOAT selection and the edit with its evidence**, so each change is traceable end-to-end rather than a bare prediction. Example grounding note: “defined in history: False; used by recipient 0x, by sender 0x; ...”. No post-hoc neural explainers (SHAP, LIME, attention) are used or claimed; none apply to a rule-based system.

6. Limitations and ethical considerations

- Evaluation validity:** 59 small, synthetic, project-author cases; the rules and cases share an author; holdout is the same style. Scores show designed behaviour on controlled cases, **not real-world accuracy** (needs real conversations and independent annotators).
- Coverage:** English only; lexicon / regex based; misses paraphrase and unmarked implicit context; false positives on idioms.
- Fairness:** no demographic or sensitive attributes in the data, so no demographic fairness analysis was done; no claim of being unbiased or fair. Recipients with non-native English or different dialects are untested.
- Trust:** sender-supplied context notes are trusted as given (system cannot verify the sender actually discussed it). LLM mode was tested only with mocked HTTP, not live APIs, and is not bit-for-bit reproducible. Implements AOT/GOAT as thresholds + a greedy edit set; it does **not** implement RLM, IDT, Cox model, APD, embeddings or fine-tuning.

7. Reproducibility and recommendations

Reproduce: clone github.com/painsparc/AOT-GOAT → python3 -m venv .venv && . .venv/bin/activate → pip install -r requirements.txt → python test_pipeline.py (28 tests) → python evaluate.py (prints metrics, writes eval_report.json) → python app.py (UI/API on :8000; or run.sh / run.bat / Docker). Fully offline and deterministic; no API key; no random seeds needed. Thresholds live in CFG at the top of aotgoat.py.

- Next steps:** collect real conversations with independent annotators and a larger holdout; retain row-level outputs for per-type / per-context subgroup audits; add paraphrase-robust detection (e.g., a contextual model) and re-run the same audit; test live LLM mode before any demo; add authentication and rate limiting before public deployment.